Gerência de Memória: Memória Virtual

Professor: Paulo Roberto de Oliveira
Disciplina: Sistemas Operacionais

Departamento de Informática
Universidade Estadual de Maringá

Introdução

- **Problema:** Programas muito grandes podem não caber na memória disponível.
- **Solução:** Dividir o programa em partes, os overlays:
  - A execução começa pelo overlay 0;
  - Quando ele termina, outro é carregado para ser executado;
  - Os overlays são mantidos em disco e sofrem swap in e out quando necessário (feito pelo SO);
  - A princípio esta divisão do programa era feita pelo programador (tarefa difícil, demorada e complexa);

Introdução

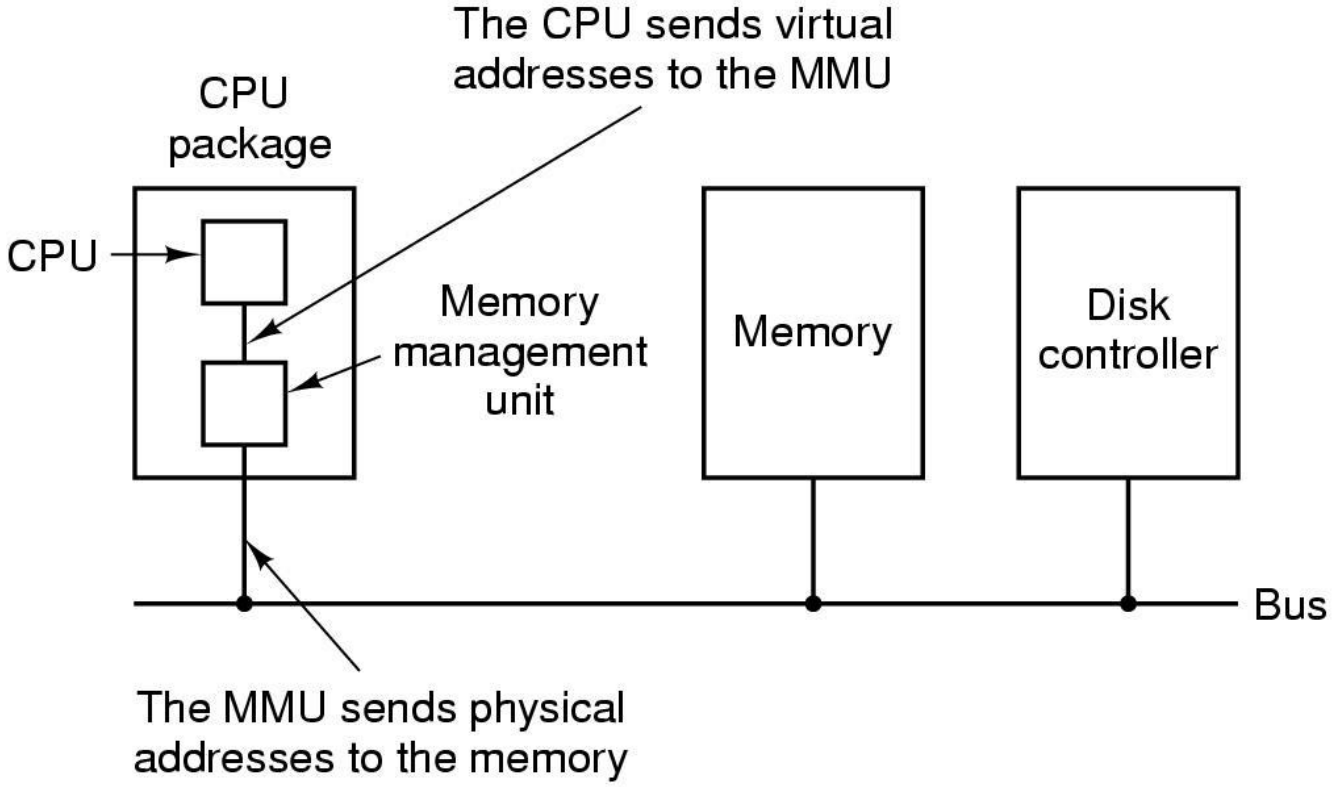
- Como deixar esta tarefa para o computador?
- Fotheringham (1961): Memória virtual!
- **Ideia:** O tamanho do programa, dados e *stack* pode exceder a memória real;
- SO mantém as partes em uso em memória e o resto em disco.

Introdução

- **Ex.:** Programa de 1 MB pode rodar em 256 KB escolhendo-se com cuidado 256 KB de seu espaço para residir em memória.
- **Ex.:** 8 programas de 1 MB podem rodar em 2 MB de memória real escolhendo-se 256 KB de cada um.
- **Obs.:** Memória virtual e multiprogramação combinam muito bem.
- Se um programa precisa fazer *swap in* de alguma parte sua, ele fica bloqueado esperando por I/O.

Paginação

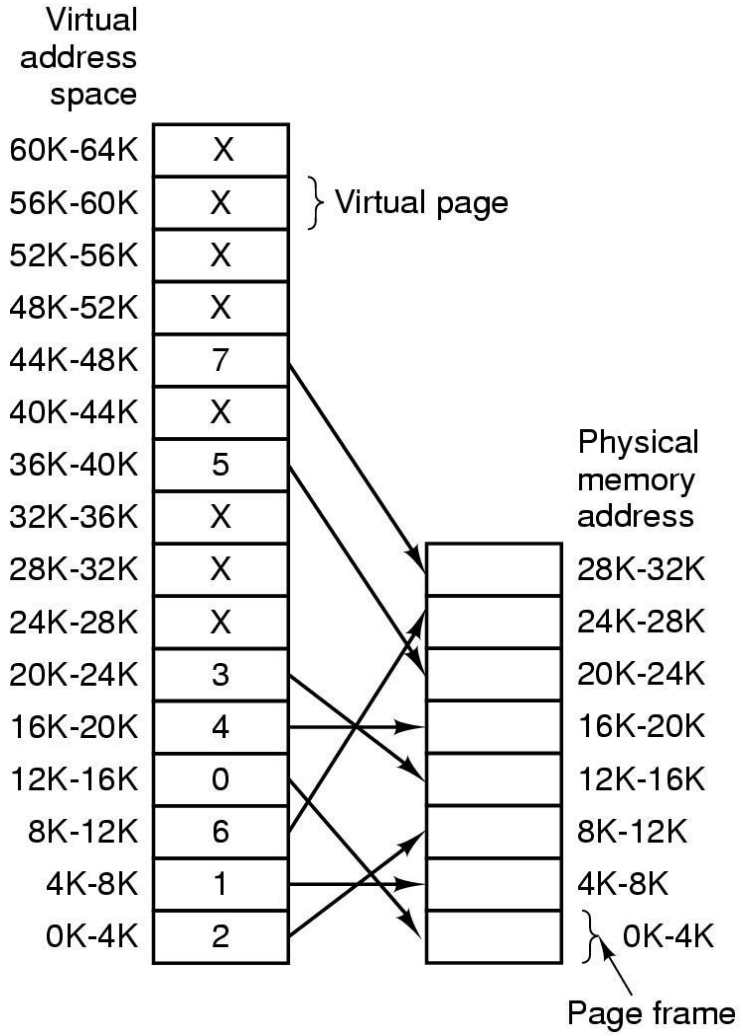
- Em qualquer computador, existe um conjunto de endereços que um programa pode produzir.



Paginação

- **Ex.:** MOVE A, $1000 - Copia conteúdo do endereço da memória 1000 para o registrador A.
  - 1000 é um endereço virtual.
  - O conjunto de endereços virtuais forma o espaço de endereçamento virtual.
  - Quando não há endereçamento virtual, o endereço gerado é colocado diretamente no *bus* para acessar uma palavra de memória real.
  - Com memória virtual, o endereço virtual não vai diretamente para o bus. Antes disso, ele passa pela Unidade de Gerência de Memória (MMU).

Paginação


Virtual address space
60K-64K
56K-60K
52K-56K
48K-52K
44K-48K
40K-44K
36K-40K
32K-36K
28K-32K
24K-28K
20K-24K
16K-20K
12K-16K
8K-12K
4K-8K
0K-4K
X
X
X
X
7
X
5
X
X
X
3
4
0
6
1
2
Virtual page
Physical memory address
28K-32K
24K-28K
20K-24K
16K-20K
12K-16K
8K-12K
4K-8K
0K-4K
Page frame

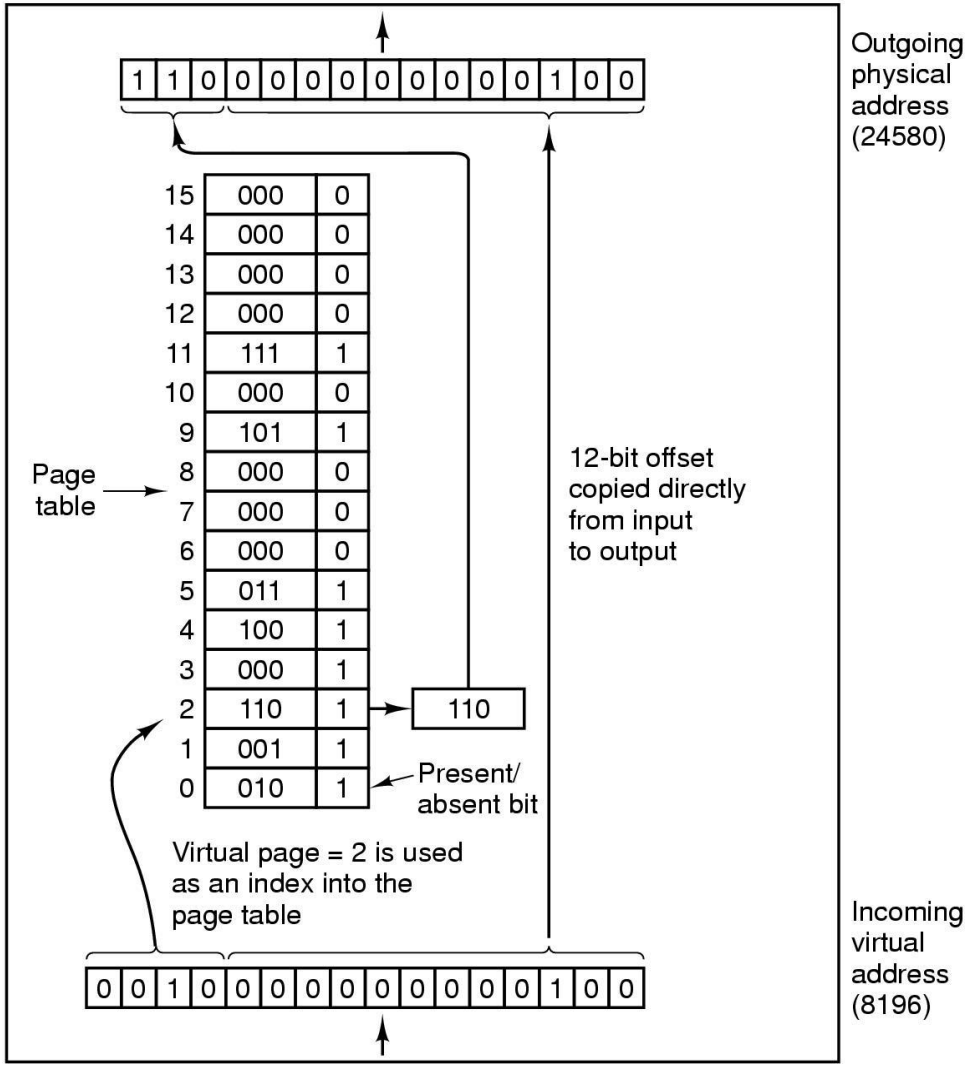
Paginação

- É preciso manter uma cópia completa do programa em disco, até os 64 KB.
- O espaço de endereçamento virtual é dividido em unidades: As páginas.
- A memória física é dividida em frames (quadros).
- Páginas e quadros são do mesmo tamanho: Valores usados são da faixa de 512 bytes a 8 KB. No exemplo são de 4 KB e temos 16 páginas virtuais e 8 quadros.

Paginação

Ideia: Usar parte do endereço virtual como índice na tabela, onde está o endereço base da moldura correspondente na memória.

O nº da página física (110) é copiado para os três bits mais significativos do endereço de saída (real), juntamente com o deslocamento.



Paginação

- **Exemplo:** Instrução MOVE A, &0
  - Endereço virtual 0 é enviado para a MMU;
  - Esse endereço cai na página 0 (0-4 KB);
  - De acordo com o mapeamento, corresponde ao quadro 2 (8-12 KB);
  - A MMU transforma o end. 0 em 8192 e coloca-o no *bus*;
  - A memória não sabe sobre a MMU. Foi solicitado o endereço 8192 e ela atende;
  - Da mesma forma, a CPU pediu o endereço 0 e recebeu.

Paginação

- Como o programa não cabe na memória real, porque ela é menor que o espaço de endereçamento, apenas algumas páginas são mapeadas em quadros. As demais ficam em disco e um *presence bit* indica se a página está na memória ou não.
- **Ex.:** MOVE A, &32780
  - 32780 é o byte 12 da página 8 (inicia em 32768);
  - A MMU descobre que a página não está mapeada (x);
  - Um trap é gerado pela CPU para o SO (page fault);
  - O SO escolhe uma página pouco usada e escreve seu conteúdo em disco;
  - Depois a página referenciada é carregada nesse quadro.

Paginação
Tabelas de Páginas

Principais questões:

1. A tabela de páginas pode ser extremamente grande:

- Computadores modernos usam endereços de 32 bits;
- Com páginas de 4 KB (12 bits), 32 bits podem endereçar pouco mais de 1 milhão de páginas ($2^{32}/2^{12} = 1.048.576$)

2. O mapeamento deve ser rápido:

- O mapeamento deve ser feito em todos os acessos;
- Uma instrução típica tem um operador e um operando de memória;
- Assim, são feitos 1, 2 ou mais acessos por instrução;
- Se uma instrução demora 10 nseg, a pesquisa na tabela de páginas deve ser feita em poucos nseg. para não se tornar um gargalo.

Paginação
Tabelas de Páginas

- As tabelas de páginas são mantidas em memória para melhor desempenho.

Exemplo: MOVE A, B

- Só referencia registradores;
- Sem paginação: Basta acessar a memória para carregar a instrução;
- Com paginação: É necessário acesso adicional para a tabela de páginas;

- Desta forma, a velocidade de execução é limitada pela taxa de carga de instruções e dados;
- A paginação pode causar perda de até 20% de desempenho!

Paginação
Memória Associativa

- **Solução:** A maior parte dos acessos é feita em poucas páginas.
- Acrescentar um hardware que faz a tradução de endereços virtuais sem o uso da tabela de páginas **(Memória Associativa)**.

- Como funciona?
  - Quando um endereço é apresentado à MMU, 1º o hardware verifica se o endereço está na memória associativa;
  - O teste é feito em paralelo para todas as entradas;
  - Se ela estiver presente, verifica-se se a proteção não é violada;
  - Se não, o frame é acessado.

Paginação
Memória Associativa

- O que acontece se o endereço não está na memória associativa?
  - A MMU verifica que o endereço não foi encontrado;
  - É feita uma pesquisa comum na tabela de páginas;
  - Uma das entradas da mem. assoc. é descartada;
  - Os dados da página acessada são copiados para essa entrada.

Análise de Desempenho da Mem. Associativa:

- 100 ns para acesso na tabela de páginas;
- 20 ns para acesso na memória associativa;
- Com 90% de acerto na memória associativa tempo:
  - Tempo médio = 0,9 x 20 + 0,1 x 100 = 28 ns.
- **Obs.:** A troca de contexto invalida a memória associativa.

Algoritmos de Substituição de Páginas

Algoritmo Ótimo

- Melhor algoritmo, mas impossível de implementar;
- Em um *Page Fault*, considerar as páginas que estão em memória;
- Para cada uma, determinar quantas instruções faltam para que seja acessada novamente;
- Isso significa determinar o fluxo dos programas e quantas instruções ainda serão executadas antes que uma página seja acessada de novo;
- Isso pode determinar que uma página será acessada em 10, 100 ou 1000 instruções;

Algoritmos de Substituição de Páginas

Algoritmo Ótimo

- Esse número é usado como um *label* para cada uma das páginas;
- Aquela que tiver o maior *label* é selecionada para ser descartada!
- **Ex.:** Uma página será acessada em 1.000.000 de instruções. Outra em 10.000.000. Descartar a última significa adiar o momento de tratar um *page fault*;
- **Obs.:** Computadores, como pessoas, tendem a adiar os eventos desagradáveis o máximo possível;
- **Avaliação:** É impossível saber quanto falta para se acessar uma página!

Algoritmos de Substituição de Páginas

Páginas Não Recentemente Usadas (NRU)

- 2 bits associados com cada página:
  - R – referenciada (Lida ou escrita);
  - M – modificada (Escrita).
- Esses bits são setados toda vez que uma página é acessada.
  - Mantidos por hardware;
  - Em cada interrupção do relógio, todos os bits R são zerados;
  - Isso é feito para distinguir entre as páginas referenciadas recentemente daquelas que não foram;

Algoritmos de Substituição de Páginas

Páginas Não Recentemente Usadas (NRU)

- Em um *page fault*, todas as páginas são divididas em 4 categorias:
  - Classe 0: não referenciada, não modificada; (R = 0, M = 0)
  - Classe 1: não referenciada, modificada; (R = 0, M = 1)
  - Classe 2: referenciada, não modificada; (R = 1, M = 0)
  - Classe 3: referenciada, modificada. (R = 1, M = 1)

- Quando a classe 1 ocorre?
- Classe 1 ocorre quando uma página classe 3 tem o bit R zerado numa interrupção de relógio.

Algoritmos de Substituição de Páginas

Páginas Não Recentemente Usadas (NRU)

- NRU remove uma página aleatoriamente da classe mais baixa.
- Preferível remover uma página modificada que uma referenciada com frequência;

- **Vantagens:** Fácil de Implementar, de entender e tem um bom desempenho.

Algoritmos de Substituição de Páginas

FIFO (First In First Out)

- **Funcionamento:**
  - O SO mantém uma lista ordenada de páginas, sendo que a 1ª é a mais velha;
  - Quando chega uma página nova, ela é inserida no fim da fila;
  - Se é preciso descartar uma página, a 1ª é escolhida!

Algoritmos de Substituição de Páginas

FIFO (First In First Out)

- **Problemas:**
  - Não considera a importância das páginas;
  - Descarta páginas antigas, mas que podem ser necessárias;
  - Mantém as páginas mais novas, mas que talvez não tenham importância;
  - Raramente usado em sua forma pura!

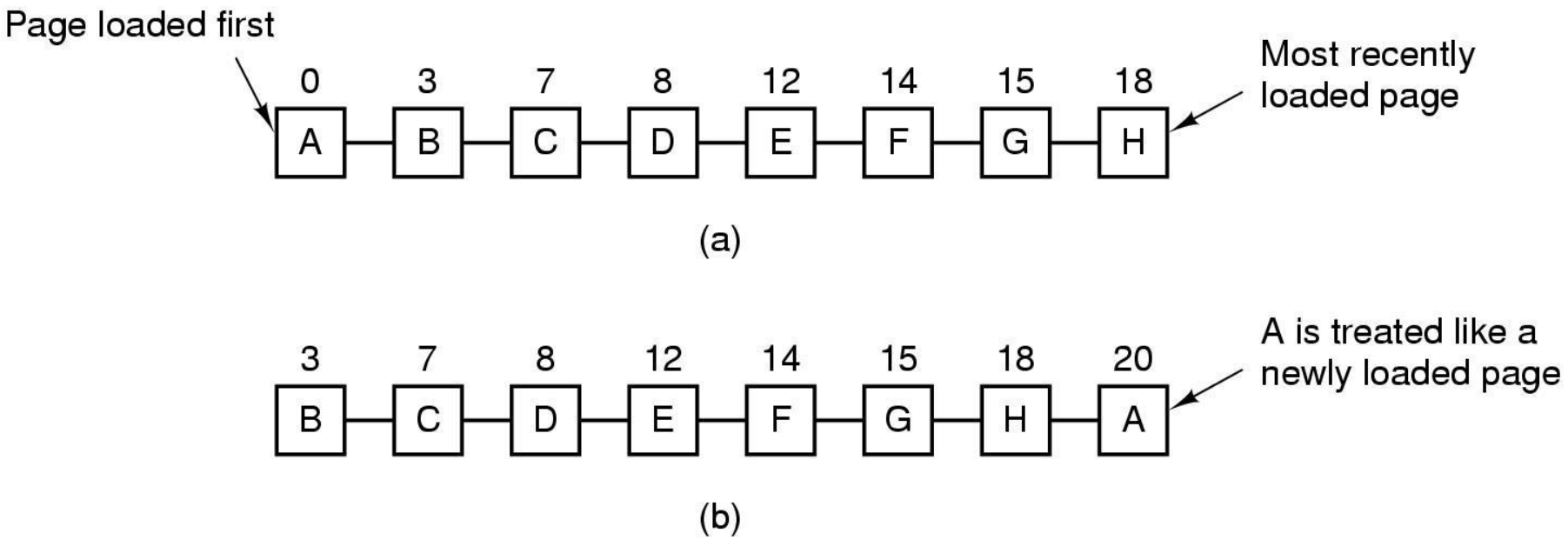
Algoritmos de Substituição de Páginas

Segunda Chance

- Modificação do FIFO para evitar o descarte de uma página pesadamente usada;
- Antes do descarte, verifica o bit R da página candidata;
- Se R = 0, então a página é antiga e não utilizada - é substituída;
- Se R = 1, o bit R é resetado e a página é colocada no final da fila (Obs.: a 1a. é a mais velha) - seu tempo de chegada é atualizado para este momento;
- O algoritmo continua pesquisando a fila;
- Se todas as páginas têm R=1 (todas foram referenciadas no último período), funciona como o FIFO.

Algoritmos de Substituição de Páginas

Segunda Chance



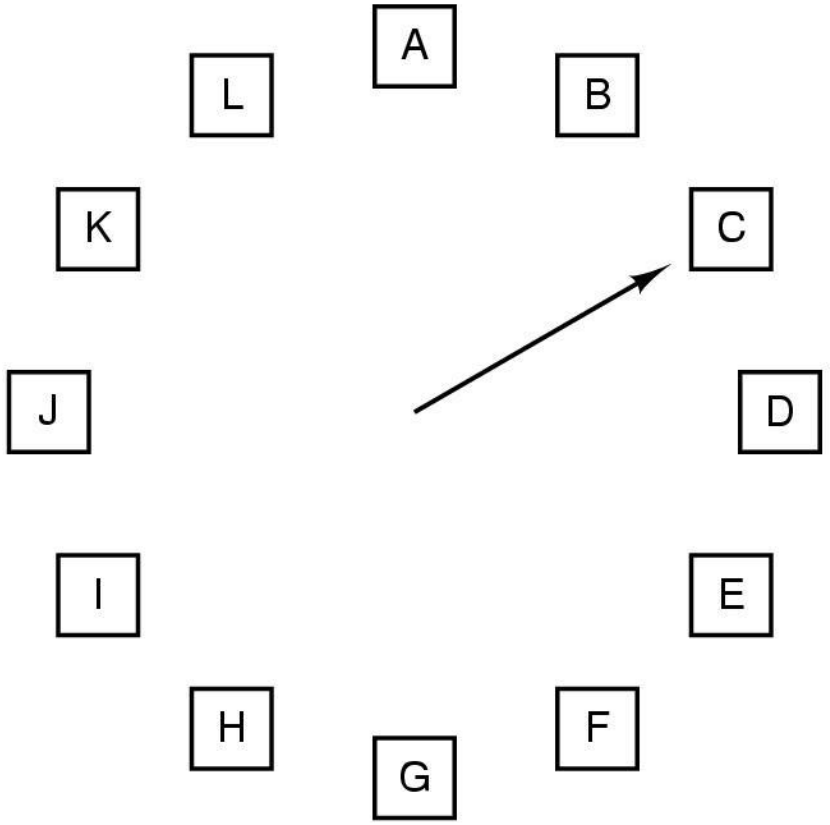
Algoritmos de Substituição de Páginas

O Relógio

- FIFO e 2a. Chance precisam manipular filas;
- O 2a. Chance precisa mudar as páginas de lugar na fila;
- Como a ordem não se altera (ordem de chegada no sistema), é melhor manter as páginas organizadas numa fila circular;
  1. Um ponteiro (ptr) aponta para o mais velho;
  2. Num page fault, se ptr->R = 0:
  3. A página é descartada;
  4. A nova entra no lugar;
  5. ptr++!

Algoritmos de Substituição de Páginas

O Relógio



When a page fault occurs, the page the hand is pointing to is inspected. The action taken depends on the R bit:

R = 0: Evict the page

R = 1: Clear R and advance hand

Algoritmos de Substituição de Páginas

LRU (Last Recently Used)

- **Observação:**
  - Páginas bastante usadas nas últimas instruções serão bastante usadas nas próximas;
  - Páginas não usadas por vários períodos provavelmente permanecerão sem utilização.

- **Estratégia:**
  - Num page fault, procurar pela página que não é usada há mais tempo.

Algoritmos de Substituição de Páginas

LRU (Last Recently Used)

- Como implementar este algoritmo?
- 1º: Manter todas as páginas numa lista encadeada;
- Esta lista precisa ser ordenada pela hora de acesso:
  - A página usada mais recentemente fica na cabeça da lista;
  - Aquela que não é usada a mais tempo fica no final da lista.
- O problema é manter a fila ordenada!

Algoritmos de Substituição de Páginas

LRU (Last Recently Used)

- Quer dizer, é preciso ordenar a fila em cada acesso - a instrução atual coloca a página acessada na cabeça da fila:
  - 1º a página da instrução torna-se a 1a.;
  - Depois, a página do 1º operando;
  - Por último, a página do 2º e 3º!

- Esse processo implica em atualizar a fila em cada acesso;
- A movimentação de páginas é muito cara!!!

Algoritmos de Substituição de Páginas

LRU (Last Recently Used)

- Implementação em hardware:
  - Contador atualizado em cada acesso!
  - Num page fault, o hardware procura o menor contador (LRU)!

- 2a. solução:
  - Para uma máquina de n frames, o hardware LRU mantém uma matriz de n x n bits - inicialmente todos zerados;
  - Em um acesso a uma página, todos os bits da linha são setados e todos os bits da coluna são zerados;
  - A qualquer momento, a linha com o menor valor binário corresponde à página LRU!

Algoritmos de Substituição de Páginas

LRU (Last Recently Used)

Ex.: Um sistema com 4 páginas (0-3); Acessos às páginas 0 1 2 3 2 1 0 3 2 3.

(a)

	Page 0	1	2	3
0	0	1	1	1
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0

(b)

Page 0	1	2	3
0	0	1	1
1	0	1	1
0	0	0	0
0	0	0	0

(c)

Page 0	1	2	3
0	0	0	1
1	0	0	1
1	1	0	1
0	0	0	0

(d)

Page 0	1	2	3
0	0	0	0
1	0	0	0
1	1	0	0
1	1	1	0

(e)

Page 0	1	2	3
0	0	0	0
1	0	0	0
1	1	0	1
1	1	0	0

(f)

0	0	0	0
1	0	1	1
1	0	0	1
1	0	0	0

(g)

0	1	1	1
0	0	1	1
0	0	0	1
0	0	0	0

(h)

0	1	1	0
0	0	1	0
0	0	0	0
1	1	1	0

(i)

0	1	0	0
0	0	0	0
1	1	0	1
1	1	0	0

(j)

0	1	0	0
0	0	0	0
1	1	0	0
1	1	1	0

Dúvidas?